PaXini

# API Reference Python



PaXini Tech

# GMH15



# CONTENTS

PaXini


1. Control Class DexH15Control — 01

2. Camera Control Class DexH15Camera — 34

3. Kinematics Parsing Class DexH15Kinematic — 37

4. Joint Order Definition — 40

5. Data Structures — 41

6. Enumeration Types — 44

# 1. Control Class DexH15Control

**Responsible for core operations of the dexterous hand, including serial communication, motor control, sensor reading, force control, action management, fault diagnosis, and firmware upgrade.**

**Constructor: ctrl = DexH15Control(), no parameters.**

## 1.1 connectModbusAuto

- **Function Overview**

  Automatically connects to the first dexterous hand device on the current serial port. Suitable for single-device scenarios; the system automatically completes the device discovery and connection process.

- **Function Prototype**

```
def connectModbusAuto(self, hand_port_num: str) -> tuple[bool, int]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| hand_port_num | str | Input | Serial port number the dexterous hand is connected to (e.g., "/dev/ttyUSB0") |

- **Return Value**

  On success, returns a tuple[bool, int] containing two elements:

  - result[0] (bool): Whether the connection succeeded
  - result[1] (int): The first device address returned after a successful connection

- **Special Notes**

  If an error occurs during execution, an exception is thrown.

## 1.2 openModbusDevice

- **Function Overview**

  Opens the specified serial port device and establishes a Modbus communication connection.

- **Function Prototype**

```
def openModbusDevice(self, hand_port_num: str, baud_rate: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| hand_port_num | str | Input | Serial port number the dexterous hand is connected to |
| baud_rate | int | Input | Baud rate for serial device communication: 4000000, 921600, 460800, 230400,115200, 57600 |

- **Return Value**

  - True: Opened successfully
  - False: Failed to open



## 1.3 openUsbDevice

- **Function Overview**

  Opens the USB serial device and establishes a connection, automatically obtaining the slave address.

- **Function Prototype**

```
def openUsbDevice(self, hand_port_num: str) -> tuple[bool, int]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| hand_port_num | str | Input | USB device number |

- **Return Value**

  Returns result: tuple[bool, int]:

  - result[0] (bool): Whether the device was opened successfully
  - result[1] (int): The slave address returned after a successful connection

## 1.4 scanModbusDevices

- **Function Overview**

  Scans for all reachable device addresses on the opened serial port.

- **Function Prototype**

```
def scanModbusDevices(self, hand_port_num: str) -> list[int]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| hand_port_num | str | Input | Serial port number the dexterous hand is connected to |

- **Return Value**

  Returns the set of all reachable device addresses on the opened serial port (list[int]).

## 1.5 initModbusDevice

- **Function Overview**

  Initializes the dexterous hand Modbus device configuration.

- **Function Prototype**

```
def initModbusDevice(self, slave_address: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  - 1: Initialization succeeded
  - 0: Initialization failed

## 1.6 disconnectModbus

- **Function Overview**

  Closes the current Modbus serial connection and releases serial port resources.

- **Function Prototype**

```
def disconnectModbus(self) -> bool
```

- **Parameter Description**

  None

- **Return Value**

  - True: Disconnected successfully
  - False: Failed to disconnect



## 1.7 disconnectUsb

- **Function Overview**

  Closes the USB device connection and releases related resources.

- **Function Prototype**

```
def disconnectUsb(self) -> bool
```

- **Parameter Description**

  None

- **Return Value**

  - True: Operation completed

## 1.8 isModbusDeviceConnected

- **Function Overview**

  Checks the Modbus connection status of the dexterous hand at the specified slave address.

- **Function Prototype**

```
def isModbusDeviceConnected(self, slave_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  - True: Device is connected
  - False: Device is not connected

## 1.9 setModbusBaudrate

- **Function Overview**

Sets the communication baud rate of the Modbus slave.

- **Function Prototype**

```
def setModbusBaudrate(self, slave_address: int, baud_rate: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| baud_rate | int | Input | Target baud rate: 4000000, 921600, 460800, 230400, 115200, 57600 |

- **Return Value**

- True: Set successfully
- False: Failed to set

- **Special Notes**

The baud rate must be set before using the serial port, and the device must be restarted for the setting to take effect. All devices mounted on the same serial port must use the same baud rate.

## 1.10 getModbusBaudrate

- **Function Overview**

Gets the current communication baud rate of the Modbus slave.

- **Function Prototype**

```
def getModbusBaudrate(self, slave_address: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns the current baud rate value of the slave device (int).



## 1.11 setModbusDeviceAddress

- **Function Overview**

Modifies the device address of the Modbus slave.

- **Function Prototype**

```
def setModbusDeviceAddress(self, slave_address: int, new_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Current slave address (0x01 ~ 0xF7) |
| new_address | int | Input | New slave address (0x01 ~ 0xF7) |

- **Return Value**

- True: Set successfully
- False: Failed to set

- **Special Notes**

Gets the Modbus communication configuration of the current slave device.

## 1.12 getModbusConfig

- **Function Overview**

Gets the Modbus communication configuration of the current slave device.

- **Function Prototype**

```
def getModbusConfig(self, slave_address: int) -> ModbusConfig
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a ModbusConfig struct containing the baud_rate (baud rate) and slave_id (slave ID) fields.

## 1.13 enableMotor

- **Function Overview**

Enables (powers on and activates) all motors of the specified slave.

- **Function Prototype**

```
1  def enableMotor(self, slave_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

- True: Disabled successfully
- False: Failed to enable

## 1.14 disableMotor

- **Function Overview**

Disables (powers off ) all motors of the specified slave.

- **Function Prototype**

```
1  def disableMotor(self, slave_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

- True: Disabled successfully
- False: Failed to disable



## 1.15 isMotorEnabled

- **Function Overview**

Queries the current enable status of the specified slave's motors.

- **Function Prototype**

```
1  def isMotorEnabled(self, slave_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

- True: Motors are enabled
- False: Motors are not enabled

## 1.16 initMotorPosition

- **Function Overview**

Initializes all motor positions, returning the dexterous hand to its initial zero position.

- **Function Prototype**

```
1  def initMotorPosition(self, slave_address: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

- 1: Initialization succeeded
- 0: Initialization failed

## 1.17 getMotorMaxVelocityConfig

- **Function Overview**

Sets the control mode of the motors. The default mode is position mode; it can be switched to current mode or velocity mode as needed.

- **Function Prototype**

```
1  def setMotorControlMode(self, slave_address: int, mode: ControlMode) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| mode | ControlMode | Input | Motor control mode enum value |

- **ControlMode Enum Value Description**

| Enum Value | Description |
|---|---|
| NONE | Invalid value |
| CURRENT_CONTROL_MODE | Current-loop control mode |
| VELOCITY_CONTROL_MODE | Velocity-loop control mode |
| POSITION_CONTROL_MODE | Position-loop control mode (default) |

- **Return Value**

- True: Set successfully
- False: Failed to set

## 1.18 getMotorControlMode

- **Function Overview**

Gets the current control mode of the motors.

- **Function Prototype**

```
def getMotorControlMode(self, slave_address: int) -> ControlMode
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a ControlMode enum value:

| Enum Value | Description |
|---|---|
| NONE | Invalid value (appears only when retrieval fails) |
| CURRENT_CONTROL_MODE | Current-loop control mode |
| VELOCITY_CONTROL_MODE | Velocity-loop control mode |
| POSITION_CONTROL_MODE | Position-loop control mode |

## 1.19 setJointPositionsAngle (All Joints)

- **Function Overview**

Controls the target angles (normalized angles) of all 11 finger joints. For joint order, see "4. Joint Order Definition".

- **Function Prototype**

```
def setJointPositionsAngle(self, slave_address: int, joint_angles: list[float]) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int16_t | Input | Slave address |
| joint_angles | list[float] | Input | Set of normalized angle values for the 11 adjustable joints |

- **Return Value**

- 1: Set successfully
- 0: Failed to set



## 1.20 setJointPositionsAngle (Specified Finger)

- **Function Overview**

Sets the normalized joint angles of a single specified finger.

- **Function Prototype**

```
def setJointPositionsAngle(self, slave_address: int, finger_id: FingerType, joint_angles: list[float]) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int16_t | Input | Slave address |
| finger_id | FingerType | Input | Enum value of the target finger |
| joint_angles | list[float] | Input | Set of normalized angle values for the specified finger |

- **FingerType Enum Value Definition**

| Enum Value | Value | Description |
|---|---|---|
| INVALID_FINGER | 0 | Invalid value |
| INDEX_FINGER | 1 | Index finger |
| MIDDLE_FINGER | 2 | Middle finger |
| RING_FINGER | 3 | Ring finger |
| PINKY_FINGER | 4 | Pinky finger |
| THUMB_FINGER | 5 | Thumb |

- **Return Value**

- 1: Set successfully
- 0: Failed to set

## 1.21 getJointPositionsAngle (All Joints)

- **Function Overview**

Obtain the current normalized angles of all finger joints. When is_all_joint is false, 11 adjustable joint angles are acquired; when it is true, 4 additional distal phalanx (DIP) joints are returned, resulting in a total of 15 joint angles being obtained.

- **Function Prototype**

```
def getJointPositionsAngle(self, slave_address: int, is_all_joint: bool = False) -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| is_all_joint | bool | Input | Whether to obtain all joint angles; the default value is False (11 adjustable joints are obtained by default) |

- **Return Value**

返回 tuple[int, list[float]]:
- result[0] int: 1 for success / 0 for failure
- result[1] list[float]: When is_all_joint= False, the length is 11; when is_all_joint= True, the length is 15 (the joint sequence is the same as that in Section 1.32 of the C++ version)

## 1.22 getJointPositionsAngle (specified finger)

- **Function Overview**

Obtain the current joint angles of a single specified finger. When is_all_joint is false, this finger returns 2 joints (3 for the thumb); when it is true, an additional distal phalanx (DIP) joint of this finger will be returned as well.

- **Function Prototype**

```
def getJointPositionsAngle(self, slave_address: int, finger_id: FingerType,
                           is_all_joint: bool = False) -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter Name | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| finger_id | FingerType | Input | Enumeration value of the target finger |
| is_all_joint | bool | Input | Whether to obtain all joint angles of this finger, default: False |

- **Return Value**

Returns tuple[int, list[float]]:

- result[0] int: 1 for success / 0 for failure
- result[1] list[float]: Joint angle values of this finger

## 1.23 setMotorTargetPosition (All Motors)

- **Function Overview**

Uniformly sets the target positions of all motors via the motor position parameter struct.

- **Function Prototype**

```
def setMotorTargetPosition(self, slave_address: int, target_position: Dex15MotorPosition) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int16_t | Input | Slave address |
| target_position | Dex15MotorPosition | Input | Target position values for the 7 motors |

- **Return Value**

- 1: Set successfully
- 0: Failed to set



## 1.24 setMotorTargetPosition (Specified Motor)

- **Function Overview**

Sets the target position of a single specified motor.

- **Function Prototype**

```
def setMotorTargetPosition(self, slave_address: int, motor_id: FingerMotor, target_position: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| motor_id | FingerMotor | Input | Enum index value of the target motor (see 6.4 FingerMotor) |
| target_position | int | Input | Target motor position value |

- **Return Value**

- 1: Set successfully
- 0: Failed to set

## 1.25 getMotorPosition

- **Function Overview**

Gets the current position information of all motors.

- **Function Prototype**

```
def getMotorPosition(self, slave_address: int) -> Dex15MotorPosition
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a Dex15MotorPosition struct containing the current position information of all motors (motor1_pos ~ motor7_pos).

## 1.26 getJointMagneticEncoder

- **Function Overview**

Gets the raw magnetic encoder data of all joints of the current slave device.

- **Function Prototype**

```
def getJointMagneticEncoder(self, slave_address: int) -> list[int]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

1: Set successfully
0: Failed to set

## 1.27 setMotorTargetVelocity (All Motors)

- **Function Overview**

In velocity mode, uniformly sets the target velocity of all motors, used to control the overall motion speed of thehand.

- **Function Prototype**

```
def setMotorTargetVelocity(self, slave_address: int, target_speed: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| target_speed | int | Input | Uniform target velocity for all motors, range [-1000, 1000] |

- **Return Value**

1: Set successfully
0: Failed to set



## 1.28 setMotorTargetVelocity (Specified Motor)

- **Function Overview**

In velocity control mode, sets the target velocity of a single specified motor.

- **Function Prototype**

```
def setMotorTargetVelocity(self, slave_address: int, motor_id: FingerMotor, target_speed: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| motor_id | FingerMotor | Input | Enum index value of the target motor (see 6.4 FingerMotor) |
| target_speed | int | Input | Target velocity value, range [-1000, 1000] |

- **Return Value**

1: Set successfully
0: Failed to set

## 1.29 getMotorActualVelocity

- **Function Overview**

Gets the actual running velocity of all current motors.

- **Function Prototype**

```
def getMotorActualVelocity(self, slave_address: int) -> tuple[int, Dex15MotorSpeed]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns result: tuple[int, Dex15MotorSpeed]:

- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (Dex15MotorSpeed): Actual velocity of the 7 motors, range [-1000, 1000]

## 1.30 setMotorMaxVelocity

- **Function Overview**

Sets the maximum velocity limit of all motors.

- **Function Prototype**

```
def setMotorMaxVelocity(self, slave_address: int, max_speed: Dex15MotorSpeed) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| max_speed | Dex15MotorSpeed | Input | Maximum velocity configuration for each motor, range [0, 1000] |

- **Return Value**

- True: Set successfully
- False: Failed to set

## 1.31 getMotorMaxVelocityConfig

- **Function Overview**

Gets the maximum velocity configuration of all motors of the current slave device.

- **Function Prototype**

```
def getMotorMaxVelocityConfig(self, slave_address: int) -> Dex15MotorSpeed
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a Dex15MotorSpeed struct containing the maximum velocity configuration of all motors.



## 1.32 setMotorTargetCurrent (All Motors)

- **Function Overview**

In current mode, sets the target current of all motors. The larger the target current value, the stronger the grip force of the hand.

- **Function Prototype**

```
def setMotorTargetCurrent(self, slave_address: int, target_current: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| target_current | int | Input | Uniform target current for all motors, range [-1000, 1000] |

- **Return Value**

- 1: Set successfully
- 0: Failed to set

## 1.33 setMotorTargetCurrent (Specified Motor)

- **Function Overview**

In current mode, sets the target current of a single specified motor.

- **Function Prototype**

```
def setMotorTargetCurrent(self, slave_address: int, motor_id: FingerMotor, target_current: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| motor_id | FingerMotor | Input | Enum index value of the target motor (see 6.4 FingerMotor) |
| target_current | int | Input | Target current value, range [-1000, 1000] |

- **Return Value**

- 1: Set successfully
- 0: Failed to set

## 1.34 getMotorActualCurrent

- **Function Overview**

Gets the actual current values of all current motors.

- **Function Prototype**

```
def getMotorActualCurrent(self, slave_address: int) -> tuple[int, Dex15MotorCurrent]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns tuple[int, Dex15MotorCurrent]:

- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (Dex15MotorCurrent): Actual current of the 7 motors, range [-1000, 1000]

## 1.35 setMotorMaxCurrent

- **Function Overview**

Sets the maximum current of all motors, used to limit the peak grip force.

- **Function Prototype**

```
def setMotorMaxCurrent(self, slave_address: int, max_current: Dex15MotorCurrent) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| max_current | Dex15MotorCurrent | Input | Maximum current configuration for each motor, range [0, 1000] |

- **Return Value**

- True: Set successfully
- False: Failed to set



## 1.36 getMotorMaxCurrentConfig

- **Function Overview**

Gets the maximum current configuration of all motors of the current slave device.

- **Function Prototype**

```
def getMotorMaxCurrentConfig(self, slave_address: int) -> Dex15MotorCurrent
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a Dex15MotorCurrent struct containing the maximum current configuration of all motors.

## 1.37 getFingerTactile [deprecated]

- **Function Overview**

[Deprecated] This function is no longer maintained; please use getFingerResultantForce instead. Gets the resultant force information from all finger tactile sensors. The hand has a total of 5 sensors, each returning a three-axis resultant force.

- **Function Prototype**

```
def getFingerTactile(self, slave_address: int) -> list[ForcePoint]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a list containing 5 ForcePoint objects, each ForcePoint containing:

| Field | Type | Description |
|---|---|---|
| x | int | Sensor X-axis force component, unit 0.1N |
| y | int | Sensor Y-axis force component, unit 0.1N |
| z | int | Sensor Z-axis force component, unit 0.1N |

## 1.38 getFingerResultantForce

- **Function Overview**

Gets the resultant force information from all finger tactile sensors. The hand has a total of 5 sensors, each returning a three-axis resultant force.

- **Function Prototype**

```
def getFingerResultantForce(self, slave_address: int) -> list[ForcePoint]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a list containing 5 ForcePoint objects, each with x, y, z three-axis components, unit 0.1N.

## 1.39 getFingerTactileDetail [deprecated]

- **Function Overview**

[Deprecated] This function is no longer maintained; please use getFingerDistributedForce instead. Gets the detailed distributed force information from all finger tactile sensors.

- **Function Prototype**

```
def getFingerTactileDetail(self, slave_address: int) -> list[ForcePoint]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a list containing 327 ForcePoint objects. The distribution rules are as follows:

| Finger | Number of Distributed Force Points |
|---|---|
| Index finger | 50 |
| Middle finger | 50 |
| Ring finger | 50 |
| Pinky finger | 50 |
| Thumb | 127 |

Each ForcePoint contains x, y, z three-axis components, unit 0.1N.



## 1.40 getFingerDistributedForce

- **Function Overview**

Gets the distributed tactile force of all fingers, with the same semantics as getFingerTactileDetail. Gets the detailed distributed force information from all finger tactile sensors.

- **Function Prototype**

```
def getFingerDistributedForce(self, slave_address: int) -> list[ForcePoint]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a list of distributed force points (327 ForcePoint objects total), each with x, y, z three-axis components, unit 0.1N.

## 1.41 getFingerTactileData

- **Function Overview**

Retrieves the resultant force, distributed force, and temperature data of a single specified finger in one call.

- **Function Prototype**

```
def getFingerTactileData(self, slave_address: int, finger_id: FingerType) -> tuple[ForcePoint, list[ForcePoint], FingerTemp]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| finger_id | FingerType | Input | Enum type value of the target finger (see 1.20) |

- **Return Value**

Returns tuple[ForcePoint, list[ForcePoint], FingerTemp]:

- result[0] (ForcePoint): Resultant force information from the finger's tactile sensor
- result[1] (list[ForcePoint]): Distributed force information from the finger's tactile sensor
- result[2] (FingerTemp):Sensor temperature data of the finger

## 1.42 getAllTactileData

- **Function Overview**

Retrieves the resultant force, distributed force, and temperature data of all fingers in one call.

- **Function Prototype**

```
def getAllTactileData(self, slave_address: int) -> tuple[list[ForcePoint], list[ForcePoint], list[FingerTemp]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns result: tuple[list[ForcePoint], list[ForcePoint], list[FingerTemp]]:
- result[0] (list[ForcePoint]): Resultant force data of all fingers (5 items)
- result[1] (list[ForcePoint]): Distributed force data of all fingers (327 items)
- result[2] (list[FingerTemp]): Temperature data of all fingers (5 items)

## 1.43 getPressPositionForce

- **Function Overview**

Gets the set of position force points of the current pressing force of all fingers, in units of 0.1N.

- **Function Prototype**

```
def getPressPositionForce(self, slave_address: int) -> list[PressForcePoint]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns list[PressForcePoint], with the following fields per item:

| Field | Type | Description |
|---|---|---|
| x / y / z | int | Coordinate position |
| force_value | int | Force magnitude, unit 0.1N |
| model_name | str | Force magnitude, unit 0.1N |



## 1.44 getSDKVersion

- **Function Overview**

Gets the temperature data of all finger sensors. The hand has a total of 5 sensors, returning 5 temperature readings.

- **Function Prototype**

```
def getTemperature(self, slave_address: int) -> list[FingerTemp]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

Returns a list containing 5 FingerTemp objects, each FingerTemp containing:

| Field | Type | Description |
|---|---|---|
| finger_name | str | Finger name |
| sensor_ID | int | Sensor ID |
| temperature | float | Sensor temperature value (unit ° C) |

## 1.45 calibrateSensor

- **Function Overview**

Calibrates the tactile sensors.。

- **Function Prototype**

```
def calibrateSensor(self, slave_address: int) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int16_t | Input | Slave address |

- **Return Value**

- True: Calibration succeeded
- False: Calibration failed

## 1.46 setCommonForceCtlParam

### • Function Overview

Configures the initial position and control parameters for common force control. After configuration, use computeCommonForceTargetCommand to compute the target command for each motion.

### • Function Prototype

```
1  def setCommonForceCtlParam(self, slave_address: int, joint_angles: list[float], param: ForceCtlParam) -> int
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| joint_angles | list[float] | Input | Initial dexterous hand position for common force control; pass thenormalized angle values of the 11 adjustable joints |
| param | ForceCtlParam | Input | Force control parameter configuration (enable flag + impedance factor+ reference force factor); default parameters are used if not set |

### • Return Value

- 1: Configuration succeeded
- 0: Configuration failed

### • Special Notes

- The parameter configurations for common force control and tangential force control override each other, and the two cannot be set and used simultaneously. After being overridden, parameters must be set again before target positions can be computed.
- When the current position of the dexterous hand differs from the initial position joint_angles required for force control, you need to sleep and wait as appropriate for the dexterous hand to reach the initial position.

## 1.47 computeCommonForceTargetCommand

### • Function Overview

In common force control mode, computes the target joint angle command for the next motion based on the current force sensor data.

### • Function Prototype

```
1  def computeCommonForceTargetCommand(self, slave_address: int) -> tuple[int, list[float]]
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

### • Return Value

Returns result: tuple[int, list[float]]:

- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (list[float]): Normalized angle command for the next motion, containing data for the 11 adjustable joints



## 1.48 setTangentialForceCtlParam

### • Function Overview

Configures the initial position and control parameters for tangential force control. After configuration, use computeTangentialForceTargetCommand to compute the target command for each motion.

### • Function Prototype

```
1  def setTangentialForceCtlParam(self, slave_address: int, joint_angles: list[float], param: TanForceCtlParam) -> int
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| joint_angles | list[float] | Input | Initial dexterous hand position for tangential force control; pass thenormalized angle values of the 11 adjustable joints |
| param | TanForceCtlParam | Input | Tangential force control parameter configuration (includingtangential force gain); default parameters are used if not set |

### • Return Value

- 1: Configuration succeeded
- 0: Configuration failed

### • Special Notes

- The parameter configurations for common force control and tangential force control override each other, and the two cannot be set and used simultaneously. After being overridden, parameters must be set again before target positions can be computed.
- When the current position of the dexterous hand differs from the initial position joint_angles required for force control, you need to sleep and wait as appropriate for the dexterous hand to reach the initial position.

## 1.49 computeTangentialForceTargetCommand

### • Function Overview

In tangential force control mode, computes the target joint angle command for the next motion based on the current force sensor data.

### • Function Prototype

```
1  def computeTangentialForceTargetCommand(self, slave_address: int) -> tuple[int, list[float]]
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int16_t | Input | Slave address |

### • Return Value

Returns result: tuple[int, list[float]]:

- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (list[float]): Normalized angle command for the next motion, containing data for the 11 adjustable joints

## 1.50 calculateRealAngle

- **Function Overview**

Computes the corresponding real physical angle (unit: degrees) from the normalized angle values.

- **Function Prototype**

```
def calculateRealAngle(self, slave_address: int, normalize_angle: list[float]) -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| normalize_angle | list[float] | Input | Set of normalized angle values for the 11 adjustable joints (for ranges see the Normalized Range column in 4. Joint Order Definition) |

- **Return Value**

Returns result: tuple[int, list[float]]:
- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (list[float]): Set of computed real angle values for the 11 joints (unit: ° )

## 1.51 calculateNormalizeAngle

- **Function Overview**

Computes the corresponding normalized angles from the real physical angle values.

- **Function Prototype**

```
def calculateNormalizeAngle(self, slave_address: int, real_angle: list[float]) -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| real_angle | list[float] | Input | Set of real angle values for the 11 adjustable joints (unit: ° ) |

- **Return Value**

Returns result: tuple[int, list[float]]:
- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (list[float]): Set of computed normalized angle values for the 11 adjustable joints



## 1.52 saveCustomizedAction (Specified Action Frame)

- **Function Overview**

Saves the specified motor position data as a user-customized action.

- **Function Prototype**

```
def saveCustomizedAction(self, slave_address: int, index: int, action: Dex15MotorPosition) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| index | int | Input | Customized action sequence number, range [1, 20] |
| action | Dex15MotorPosition | Input | Motor position data for the 7 motors to be saved |

- **Return Value**

- 1: Saved successfully
- 0: Failed to save

## 1.53 saveCustomizedAction (Current Position)

- **Function Overview**

Directly saves the current real-time position of the dexterous hand as a customized action with the specified number.

- **Function Prototype**

```
def saveCustomizedAction(self, slave_address: int, index: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| index | int | Input | Customized action sequence number, range [1, 20] |

- **Return Value**

- 1: Saved successfully
- 0: Failed to save

## 1.54 saveCustomizedAction（关节角度）

- **Function Overview**

Computes the motor positions from the specified set of normalized joint angles and saves them as a customized action.

- **Function Prototype**

```
1  def saveCustomizedAction(self, slave_address: int, index: int, joint_angle_list: list[float]) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| index | int | Input | Customized action sequence number, range [1, 20] |
| joint_angle_list | list[float] | Input | Set of real joint angles to be saved (11 adjustable joints) |

- **Return Value**

- 1: Saved successfully
- 0: Failed to save

## 1.55 getActionPosition

- **Function Overview**

Loads and reads the motor position information corresponding to a built-in or customized action sequence.

- **Function Prototype**

```
1  def getActionPosition(self, slave_address: int, index: int, is_customized: bool = True) -> tuple[int, Dex15MotorPosition]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| index | int | Input | Action sequence number (range [1, 20] when is_customized=True; range [1, 10]when is_customized=False) |
| is_customized | bool | Input | True: customized action; False: official built-in action (default True) |

- **Return Value**

Returns tuple[int, Dex15MotorPosition]:
- result[0] (int): Status code, 1 for success / 0 for failure
- result[1] (Dex15MotorPosition): Retrieved motor position data



## 1.56 runAction

- **Function Overview**

Executes the built-in or customized action with the specified number.

- **Function Prototype**

```
1  def runAction(self, slave_address: int, index: int, is_customized: bool = True) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| index | int | Input | Action sequence number (range [1, 20] when is_customized=True; range [1, 10] when is_customized=False) |
| is_customized | bool | Input | True: customized action; False: official built-in action (default True) |

- **Return Value**

- 1: Executed successfully
- 0: Failed to execute

## 1.57 checkHandStatus

- **Function Overview**

Checks the overall running status of the dexterous hand.

- **Function Prototype**

```
1  def checkHandStatus(self, slave_address: int) -> int
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

| Return Value | Description |
|---|---|
| 1 | Running normally |
| 2 | Fault-tolerant running state (an error has occurred but does not affect the use of core interfaces) |
| 3 | Non-operational state |

## 1.58 getFaultCode

### • Function Overview

Gets the current motor fault codes, used to diagnose the causes of motor faults.

### • Function Prototype

```
1  def getFaultCode(self, slave_address: int) -> list[FingerMotorsError]
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

### • Return Value

Returns list[FingerMotorsError], a list of fault codes, with the following fields per item:

| Field | Type | Description |
|---|---|---|
| error_code | ErrorCode | Error code enum |
| finger_name | str | Finger name |
| motor_ID | int | Motor ID |

### • ErrorCode Fault Code Reference Table

| Error Name | Error Code | Error Description | Explanation | Solution |
|---|---|---|---|---|
| OVER_VOLT | 0 | Finger moto rovervoltage | Drive supply voltage above 15V for 2 seconds | After the voltage recovers, execute the clear-error command |
| UNDER_VOLT | 1 | Finger motor undervoltage | Drive supply voltage below 10V for 2 seconds | After the voltage recovers, execute the clear-error command |
| OVER_CURRENTU | 2 | U-phase overcurrent | Excessive U-phase current | After the current returns to normal, execute the clear - error command |
| OVER_CURRENTV | 3 | V-phase overcurrent | Excessive V-phase current | After the current returns to normal, execute the clear - error command |
| OVER_CURRENTW | 4 | W-phase overcurrent | Excessive W-phase current | After the current returns to normal, execute the clear - error command |
| OVER_TEMPERATURE | 5 | Finger motor overtemperature | Motor temperature too high | After the temperature returns to the normal range, execute the clear-error command |
| ADC_CALIB | 6 | Hardware self-check error | AD static deviation +10% | Check the drive's magnetic field environment to eliminate interference, then power off and restart; if ineffective, replace the drive board |
| FOC_ERR | 7 | FOC computation error | None | None |
| OVER_IQ / OPD_IQ | 8 | IQ overcurrent | Excessive motor IQ current | After the IQ overcurrent is cleared, execute the clear - error command |
| OPD_U | 9 | U-phase open | Detected during power-on initialization | Check the motor U-phase wire; after recovery, execute the clear-error command |
| OPD_V | 10 | V-phase open | Detected during power-on initialization | Check the motor V-phase wire; after recovery, execute the clear-error command |
| OPD_W | 11 | W-phase open | Detected during power-on initialization | Check the motor W-phase wire; after recovery, execute the clear-error command |
| POS_REF_OVER | 12 | Position over upper limit | None | None |
| POS_REF_UNDER | 13 | Position over lower limit | None | None |
| ABZ_ERR | 14 | Encoder abnormality | Magnetic encoder loses count > 0.25% per revolution | Check the encoder, power off and restart; if the error persists, return to factory |
| ABZ_Z_ERR | 15 | Encoder Z-phase abnormality | Cannot find Z-phase during initialization | Check the encoder, power off and restart; if the error persists, return to factory |
| ABZ_CALIB_ERR | 16 | Encoder calibration erro | Encoder wire not connected properly or not calibrated | Check the encoder or wiring |
| SPEED_REF_OVER | 17 | Given speed over limit | None | None |
| CURRENT_REF_OVER | 18 | Given current over limit | None | None |
| JOINT_ENC_DISCONNECT | 19 | Magnetic encoder on the drive board disconnected | None | None |
| MAGN2_NEGA_WARN | 27 | Magnetic encoder shows a small negative value | Magnetic encoder value less than -180 and greater than -360 | None |
| MAGN2_NEGA_ERR | 28 | Magnetic encoder shows a large negative value | Magnetic encoder value less than -360 | None |
| NORM_POS_NEGA_WARN | 29 | Normalized travel small negative number | Travel value < 0 and > -20 | None |
| NORM_POS_NEGA_ERR | 30 | Normalized travel large negative number | Travel value < -20 | None |
| DISCONNECT | 31 | Finger motor not connected | Communication between finger drive and control board disconnected | The system will automatically clear the error and recover after reconnection |

The remaining RESERVED4 ~ RESERVED10 are reserved / internal diagnostic bits.



## 1.59 clearFaultCode

### • Function Overview

Clears the current motor fault codes.

### • Function Prototype

```
1  def clearFaultCode(self, slave_address: int) -> bool
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

### • Return Value

- True: Cleared successfully
- False: Failed to clear

### • Special Notes

Not all fault codes can be cleared; for details, refer to the "Solution" column of each error code.

## 1.60 getDeviceInfo

### • Function Overview

Retrieves all device information of the slave device—including the SN code, hardware version, firmware version, and Modbus configuration—in a single call.

### • Function Prototype

```
1  def getDeviceInfo(self, slave_address: int) -> tuple[bool, str, HardwareVersion, FirmwareVersion, ModbusConfig]
```

### • Parameter Description

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

### • Return Value

Returns tuple[bool, str, HardwareVersion, FirmwareVersion, ModbusConfig]:
- result[0] (bool): Whether the retrieval succeeded
- result[1] (str): Device SN code
- result[2] (HardwareVersion): Device hardware version information
- result[3] (FirmwareVersion): Device firmware version information
- result[4] (ModbusConfig): Device Modbus communication configuration information

## 1.61 getDeviceSN

- **Function Overview**

  Gets the device SN serial number.

- **Function Prototype**

```
1  def getDeviceSN(self, slave_address: int) -> str
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  Returns the device SN code string (str).

## 1.62 setTangentialForceCtlParam

- **Function Overview**

  Queries the left/right hand type of the dexterous hand.

- **Function Prototype**

```
1  def getHandType(self, slave_address: int) -> HandType
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  Returns a HandType enum value indicating the left/right hand type of the dexterous hand:

| Enum Value | Value | Description |
|---|---|---|
| NONE_HAND | 0 | Invalid dexterous hand type |
| LEFT_HAND | 1 | Left hand |
| RIGHT_HAND | 2 | Right hand |



## 1.63 getFirmwareVersion

- **Function Overview**

  Gets the firmware version information of the device.

- **Function Prototype**

```
1  def getFirmwareVersion(self, slave_address: int) -> FirmwareVersion
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  Returns a FirmwareVersion struct; currently only the unified_version field is available.

## 1.64 getHardwareVersion

- **Function Overview**

  Gets the hardware version information of the device.

- **Function Prototype**

```
1  def getHardwareVersion(self, slave_address: int) -> HardwareVersion
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |

- **Return Value**

  Returns a HardwareVersion struct; currently only the mainboard_hardware_version field is available.

## 1.65 getSDKVersion

- **Function Overview**

  Gets the version number of the current SDK.

- **Function Prototype**

```
1  def getSDKVersion(self) -> str
```

- **Parameter Description**

  None

- **Return Value**

  Returns the SDK version number string (str).

## 1.66 upgradeFirmware

- **Function Overview**

Upgrades the firmware of the specified slave device.

- **Function Prototype**

```
1  def upgradeFirmware(self, slave_address: int, file_path: str) -> UpgradeErrorCode
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave address |
| file_path | str | Input | Firmware file path |

- **Return Value**

Returns an UpgradeErrorCode enum value; common values:

| Enum Value | Description |
|---|---|
| UPGRADE_SUCCESS | Upgrade succeeded |
| UPGRADE_CHECK_OK | Upgrade check passed |
| UPGRADE_FAIL | Upgrade failed |
| UPGRADE_REQUEST_ERROR | Upgrade request error |
| UPGRADE_EXECUTE_ERROR | Upgrade execution error |
| UPGRADE_TYPE_ERROR | Upgrade type error |
| FILE_COUNT_ERROR | File count error |
| FILE_EXIT_ERROR | File does not exist |
| FILE_FORMAT_ERROR | File format error |
| FILE_MATCH_ERROR | File match error |
| FILE_NAME_ERROR | File name error |
| FILE_SIZE_ERROR | File size error |
| FILE_VERSION_ERROR | File version error |
| FILE_HAND_TYPE_ERROR | Hand type mismatch |
| FILE_SEND_SUCCESS | File sent successfully |
| HAND_ENABLE_ERROR | Hand enable error |
| HAND_FAULT_ERROR | Hand fault |
| PARAM_PART_ERROR | Part parameter error |
| SEND_EXTRA_ACTION_ERROR | Extra action send error |
| SEND_FILE_OPERATE_ERROR | File operation send error |
| SEND_FRAME_1_ERROR | Frame 1 send error |
| SEND_FRAME_N_ERROR | Frame N send error |
| SEND_FRAME_LAST_ERROR | Last frame send error |



## 1.67 upgradeSensor

- **Function Overview**

Upgrade the sensor firmware version. Please power on the dexterous hand again before starting the upgrade.

- **Function Prototype**

```
1  def getFirmwareVersion(self, slave_address: int, info: list[UpgradeFileAndPart]) -> UpgradeErrorCode
```

- **Parameter Description**

| Parameter Name | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave Address |
| info | list[UpgradeFileAndPart] | Input | List of sensor firmware and upgrade components; the upgrade file must be placed in the same directory as the program |

- **Description of UpgradeFileAndPart Structure**

| Field | Type | Description |
|---|---|---|
| file_path | str | Firmware file path |
| upgrade_part | list[int] | Set of sensor part numbers requiring upgrade |

- **Return Value**

Returns the UpgradeErrorCode enumeration value, common values (same as upgradeFirmware in Section 1.66):

| enumeration value | Description |
|---|---|
| UPGRADE_SUCCESS | Upgrade successful |
| UPGRADE_CHECK_OK | Upgrade check passed |
| UPGRADE_FAIL | Upgrade failed |
| UPGRADE_REQUEST_ERROR | Upgrade Request Error |
| UPGRADE_EXECUTE_ERROR | Upgrade execution error |
| UPGRADE_TYPE_ERROR | Upgrade Type Error |

### 1.68 getUpgradeInfo

- **Function Overview**

  Obtain the firmware upgrade progress information. If you need to use this interface, please create a separate thread to call this interface in a loop for polling before invoking upgradeFirmware.

- **Function Prototype**

```
1  def getFirmwareVersion(self, slave_address: int) -> list[UpgradeProcess]
```

- **Parameter Description**

| Parameter Name | Type | Input/Output | Description |
|---|---|---|---|
| slave_address | int | Input | Slave Address |

- **Return Value**

  Returns a list of UpgradeProcess, where each element contains:

| Field | Type | Description |
|---|---|---|
| upgrade_type | FirmwareUpgradeType | Upgrade Type (enumeration, see the table below) |
| part | int | Upgraded firmware component number |
| process_data | int | Upgrade progress value, range [0,100] |
| status | str | Upgrade Status Description |

- **FirmwareUpgradeType Enumeration Values**

| Enumeration value | Type | Description |
|---|---|---|
| INVALID_UPGRADE | 0 | Invalid upgrade type |
| MAINBOARD_UPGRADE | 1 | Main Control Board Upgrade |
| DRIVER_UPGRADE | 2 | Electric drive board upgrade |
| SENSOR_UPGRADE | 3 | Sensor Upgrade |



# 2. Camera Control Class DexH15Camera

**Responsible for connecting the camera device paired with the dexterous hand, capturing images, and configuring parameters.**
**Constructor: cam = DexH15Camera(), no parameters.**

### 2.1 connectCameraDevice

- **Function Overview**

  Opens and connects to the specified camera device.

- **Function Prototype**

```
1  def connectCameraDevice(self, camera_port_num: str) -> None
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| camera_port_num | str | Input | Camera device number |

- **Return Value**

  No return value (None). Throws an exception if the connection fails.

### 2.2 releaseCameraDevice

- **Function Overview**

  Releases the camera device resources and disconnects.

- **Function Prototype**

```
1  def releaseCameraDevice(self) -> None
```

- **Parameter Description**

  None

- **Return Value**

  No return value (None).

## 2.3 getFrame

- **Function Overview**

Captures the current frame image from the camera.

- **Function Prototype**

```
def getFrame(self) -> numpy.ndarray
```

- **Parameter Description**

None

- **Return Value**

Returns an image frame of type numpy.ndarray (uint8 array), with a default resolution of 640×480 and a frame rate of 30 fps. If the camera is not open, returns an empty array

## 2.4 setCameraConfig

- **Function Overview**

Sets the resolution and frame rate parameters of the camera.

- **Function Prototype**

```
def setCameraConfig(self, width: int, height: int, fps: float) -> bool
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| width | int | Input | Image resolution width (pixels) |
| height | int | Input | Image resolution height (pixels) |
| fps | float | Input | Camera frame rate (frames/second) |

- **Return Value**

- True: Configuration succeeded
- False: Configuration failed

- **Special Notes**

Only 640×480, 1280×720, and 1920×1080 resolutions are supported. The frame rate only supports 30fps at 640×480 and 1280×720 resolutions, and 60fps at 1920×1080 resolution.



## 2.5 showImage

- **Function Overview**

Displays the specified image frame in a window.

- **Function Prototype**

```
def showImage(self, imgName: str, img: numpy.ndarray) -> None
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| imgName | str | Input | Name of the display window |
| img | numpy.ndarray | Input | Image to display (uint8 array) |

- **Return Value**

No return value (None).

# 3. Kinematics Parsing Class DexH15Kinematic

Responsible for forward and inverse kinematics conversion between dexterous hand joint angles and motor positions, as well as mutual conversion between normalized angles and real angles. This class does not depend on a device connection and can be called offline.

Constructor: kin = DexH15Kinematic(), no parameters.

## 3.1 forwardKinematicParse

- **Function Overview**

  Forward Kinematics Analysis: Input 7 motor position values and 8 magnetic encoder data, then calculate and output the actual angle values of the joints. When return_joint_flag is False, 11 adjustable joint angles are output; when it is True, 4 additional distal interphalangeal (DIP) joints are output, resulting in a total of 15 joint angles.

- **Function Prototype**

```python
def forwardKinematicParse(
    self,
    hand_type: HandType,
    motor_pos: list[int],
    magnetic_encoder: list[int],
    hardware_version: str = ""
    return_joint_flag: bool = False
) -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter Name | Type | Input/Output | Description |
|---|---|---|---|
| hand_type | HandType | Input | Left- or right-hand type |
| motor_pos | Sequence[int] | Input | Set of 7 motor position values, range [0, 10000] |
| magnetic_encoder | Sequence[int] | Input | Set of 8 magnetic encoder data |
| hardware_version | str | Input | Hardware version (optional, defaults to an empty string) |
| return_joint_flag | bool | Input | Whether to return all 15 joints; the default value is False (11 adjustable joints will be returned) |

- **Return Value**

  Returns result: tuple[int, list[float]]:
  - result[0] int: 1 for success / 0 for failure
  - result[1] list[float]: When return_joint_flag= False, the length is 11; when return_joint_flag= True, the length is 15 (the joint order and angle range are the same as those in Section 3.3 of the C++ version)



## 3.2 inverseKinematicParse

- **Function Overview**

  Inverse kinematics parsing: takes the real angle values of the 11 adjustable joints as input, computes and outputs the corresponding 7 motor position values.

- **Function Prototype**

```python
def inverseKinematicParse(
    self,
    hand_type: HandType,
    real_angle: list[float],
    hardware_version: str = ""
) -> tuple[int, list[int]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| hand_type | HandType | Input | Left- or right-hand type |
| real_angle | Sequence[float] | Input | Set of real angle values for the 11 adjustable joints (unit: ° ) |
| hardware_version | str | Input | Hardware version (optional, defaults to an empty string) |

- **Return Value**

  Returns tuple[int, list[int]]:
  - result[0] (int): Status code, 1 for successful parsing / 0 for failed parsing
  - result[1] (list[int]): The parsed 7 motor position values

### 3.3 calculateRealAngle

- **Function Overview**

  **Computes the corresponding real physical angles (unit: degrees) from the set of normalized angles of the 11adjustable joints.**

- **Function Prototype**

```
def calculateRealAngle(self, normalize_angle: list[float], hardware_version: str = "") -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| normalize_angle | Sequence[float] | Input | Set of normalized angles for the 11 joints |
| hardware_version | str | Input | Hardware version (optional, defaults to an empty string) |

- **Return Value**

  Returns tuple[int, list[float]]:
  - result[0] (int): Status code, 1 for success / 0 for failure
  - result[1] (list[float]): The computed set of 11 real angles (unit: ° )

- **Special Notes**

  Difference from the Control class calculateRealAngle: the Kinematics class can be used independently of serial port control and supports dexterous hands with a specified hardware_version.

### 3.4 calculateNormalizeAngle

- **Function Overview**

  **Computes the corresponding set of normalized angles from the real physical angles of the 11 adjustable joints.**

- **Function Prototype**

```
def calculateNormalizeAngle(self, real_angle: list[float], hardware_version: str = "") -> tuple[int, list[float]]
```

- **Parameter Description**

| Parameter | Type | Input/Output | Description |
|---|---|---|---|
| real_angle | list[float] | Input | Set of real angles for the 11 joints (unit: ° ) |
| hardware_version | str | Input | Hardware version (optional, defaults to an empty string) |

- **Return Value**

  Returns result: tuple[int, list[float]]:
  - result[0] (int): Status code, 1 for success / 0 for failure
  - result[1] (list[float]): The computed set of 11 normalized angles



### 3.5 getSDKVersion

- **Function Overview**

  **Gets the version number of the current SDK.**

- **Function Prototype**

```
def getSDKVersion(self) -> str
```

- **Parameter Description**

  **None**

- **Return Value**

  Returns the SDK version number string (str).

## 4. Joint Order Definition

**The arrays all contain 11 adjustable joints, in the fixed order: index finger (2) → middle finger (2) → ring finger (2) → pinky finger (2) → thumb (3).**

| Index | Joint Identifier | Corresponding Finger | Normalized Range | Real Angle Range (° ) | Description |
|---|---|---|---|---|---|
| 0 | index_joint1 | Index finger | [0, 1000] | [0, 80] | Proximal joint, flexion/ extension |
| 1 | index_joint2 | Index finger | [0, 1000] | [0, 80] | Distal joint, flexion/extension |
| 2 | middle_joint1 | Middle finger | [0, 1000] | [0, 80] | Proximal joint, flexion/ extension |
| 3 | middle_joint2 | Middle finger | [0, 1000] | [0, 80] | Distal joint, flexion/extension |
| 4 | ring_joint1 | Ring finger | [0, 1000] | [0, 80] | Proximal joint, flexion/ extension |
| 5 | ring_joint2 | Ring finger | [0, 1000] | [0, 80] | Distal joint, flexion/extension |
| 6 | little_joint1 | Pinky finger | [0, 1000] | [0, 80] | Proximal joint, flexion/ extension |
| 7 | little_joint2 | Pinky finger | [0, 1000] | [0, 80] | Distal joint, flexion/extension |
| 8 | thumb_joint1 | Thumb | [-500, 500] | [0, 109.6] | Metacarpophalangeal dual-axis joint, lateral swing |
| 9 | thumb_joint2 | Thumb | [0, 1000] | [0, 87] | Proximal joint, flexion/ extension |
| 10 | thumb_joint3 | Thumb | [0, 1000] | [0, 120.1] | Distal joint, flexion/extension |

# 5. Data Structures

## 5.1 Dex15MotorPosition

- Position values for the 7 motors. Constructor: Dex15MotorPosition().

| Field | Type | Description |
|---|---|---|
| motor1_pos ~ motor7_pos | int | Position of motors 1~7 |

## 5.2 Dex15MotorCurrent

- Current values for the 7 motors. Constructor: Dex15MotorCurrent() (uniform initial value).

| Field | Type | Description |
|---|---|---|
| motor1_current ~ motor7_current | int | Current of motors 1~7 |

## 5.3 Dex15MotorSpeed

- Speed values for the 7 motors. Constructor: Dex15MotorSpeed() (uniform initial value).

| Field | Type | Description |
|---|---|---|
| motor1_speed ~ motor7_speed | int | Speed of motors 1~7 |

## 5.4 ForcePoint

- Tactile force point.

| Field | Type | Description |
|---|---|---|
| x / y / z | int | Coordinate position |

## 5.5 PressForcePoint

- Pressing position force point.

| Field | Type | Description |
|---|---|---|
| x / y / z | int | Coordinate position |
| force_value | int | Force magnitude |
| model_name | str | Model name |

## 5.6 FingerTemp

- Finger temperature information.

| Field | Type | Description |
|---|---|---|
| finger_name | str | Finger name |
| sensor_ID | int | Sensor ID |
| temperature | float | Temperature value (° C) |



## 5.7 FingerMotorsError

- Finger motor fault information.

| Field | Type | Description |
|---|---|---|
| error_code | ErrorCode | Error code |
| finger_name | str | Finger name |
| motor_ID | int | Motor ID |

## 5.8 ForceCtlParam

- Finger motor fault information.

| Field | Type | Range (per joint) | Description |
|---|---|---|---|
| enable_flag | list[bool] | True/False | Force control enable flag for each finger |
| impedance_factor | list[float] | [1, 100] | Impedance factor |
| reference_force_factor | list[float] | [1, 10] | Reference force factor |

## 5.9 TanForceCtlParam

- Tangential force control parameters (adds a tangential gain on top of the common force control parameters).

| Field | Type | Range (per joint) | Description |
|---|---|---|---|
| enable_flag | list[bool] | True/False | Force control enable flag for each finger |
| impedance_factor | list[float] | [1, 100] | Impedance factor |
| reference_force_factor | list[float] | [1, 10] | Reference force factor |
| tangential_force_gain | float | [1, 100] | Tangential force gain |

## 5.10 ModbusConfig

- Modbus 通信配置。

| Field | Type | Description |
|---|---|---|
| baud_rate | int | Baud rate |
| slave_id | int | Slave ID |

## 5.11 HardwareVersion

- Hardware version information.

| Field | Type | Description |
|---|---|---|
| driver_hardware_version | list[MotorVersion] | Hardware version of each motor drive board |
| mainboard_hardware_version | str | Mainboard hardware version |

### 5.12 FirmwareVersion

- Firmware version information.

| Field | Type | Description |
|---|---|---|
| unified_version | str | Unified firmware version |
| driver_version | list[MotorVersion] | This field is unavailable |
| mainboard_version | str | This field is unavailable |
| sensor_version | list[SensorVersion] | This field is unavailable |

### 5.13 MotorVersion

- Version information of a single motor.

| Field | Type | Description |
|---|---|---|
| part | int | Part number |
| part_name | str | Part name |
| version | str | Version number |

### 5.14 Firmware Upgrade-Related Structures

| Structure | Field | Description |
|---|---|---|
| FirewareFields | reserve1~3、upgrade_type、upgrade_version | Firmware file field information |
| UpgradeFileAndPart | file_path、upgrade_part: set[int] | Correspondence between the upgrade file and parts |
| UpgradeInfo | file_fields、file_path、upgrade_part | Upgrade information (including file field parsing results) |
| UpgradeProcess | part、process_data、status、upgrade_type | Upgrade progress information |
| SensorSequence | part、part_name、sequence | Sensor sequence information |
| SensorVersion | part、part_name、version | Sensor version information |



# 6. Enumeration Types

### 6.1 ControlMode (Motor Control Mode)

| Member | Description |
|---|---|
| NONE | No mode (returned when retrieval fails) |
| CURRENT_CONTROL_MODE | Current control |
| VELOCITY_CONTROL_MODE | Velocity control |
| POSITION_CONTROL_MODE | Position control (default) |

### 6.2 HandType (Hand Type)

| Member | Description |
|---|---|
| NONE_HAND | Invalid hand type |
| LEFT_HAND | Left hand |
| RIGHT_HAND | Right hand |

### 6.3 FingerType (Finger Type)

| Member | Description |
|---|---|
| INVALID_FINGER | Invalid |
| INDEX_FINGER | Index finger |
| MIDDLE_FINGER | Middle finger |
| RING_FINGER | Ring finger |
| PINKY_FINGER | Pinky finger |
| THUMB_FINGER | Thumb |

### 6.4 FingerMotor (Motor Number)

| Member | Value | Description |
|---|---|---|
| INVALID_MOTOR | 0 | Invalid |
| INDEX_MOTOR | 1 | Index finger motor |
| MIDDLE_MOTOR | 2 | Middle finger motor |
| RING_MOTOR | 3 | Ring finger motor |
| PINKY_MOTOR | 4 | Pinky finger motor |
| THUMB_MOTOR_1 | 5 | Thumb lateral-swing motor |
| THUMB_MOTOR_2 | 6 | Thumb motor 2 |
| THUMB_MOTOR_3 | 7 | Thumb motor 3 |

PaXini



# PaXini Tech


Tel: **PaXini Technology (Shenzhen) Co., Ltd.** : +86 755 2357 4593

**PaXini Technology (Shanghai) Co., Ltd.** : +86 21 5456 1536

**PaXini Tianjin Super EID Factory** : +86 22 8241 1882

Email: **sales_global@paxini.com**

Website: **www.paxini.com**

Add: **PaXini Technology (Shenzhen) Co., Ltd.**
34-35th Floor，Qiancheng Business Center, 5Haicheng Road, Bao'an District, Shenzhen City, Guangdong Province

**PaXini Technology Intelligent & Manufacture Laboratory**
Yongfeng Community, Baoan District, Shenzhen City, Guangdong Province

**PaXini Technology (Shanghai) Co., Ltd.**
Room 207-209, Block A, No. 391 Guiping Road, Xuhui District, Shanghai

**PaXini (Tianjin) Super EID Factory**
Kongtian Digital Industry Park, Hexi District, Tianjin

**PaXini Manufacture and Development Center (Suzhou)**
3rd Floor, Building A02, No. 1202, Fuxin Road, Yangshe Town, Zhangjiagang City, Suzhou City, Jiangsu Province

**PaXini Sensing Technology Research Branch (Shenzhen)**
Cloud City, Building 1, Phase 6 Wangkeyuncheng, Nanshan District, Shenzhen Guangdong Province


**Scan for more information**

**AliExpress**

**Amazon**